Completado

PREPARACION · Paso 1 de 9

Preparar ReqFlow

Prepara Workshop - Bash y tu cuenta; comprueba la plantilla antes de crear ReqFlow.

Antes de empezar

Esta preparación es común a los laboratorios y se documenta aquí por ahora. Usamos exclusivamente **Git Bash dentro de Windows Terminal**, perfil **Workshop - Bash**. Los comandos `bash` de esta guía van en esa terminal. Los bloques `text` son prompts para **Copilot Chat en VS Code, modo Agent**. El portal permite leer, copiar y descargar; no ejecuta el producto ni verifica por sí solo lo que declare el agente.

Son cuatro pasos, en este orden. Hazlos con tu usuario de Windows, el mismo con el que estudiarás:

1. Crear tu carpeta del curso y la terminal Workshop - Bash.
2. Iniciar sesión en GitHub.
3. Validar las herramientas con un solo comando.
4. Comprobar Copilot en modo Agent.

Necesitas Windows 10 u 11, conexión a internet y Git for Windows, que incluye Git Bash. Para el paso 1 abre **Git Bash** desde el menú Inicio; a partir de ahí usarás Workshop - Bash.

1. Crear tu carpeta del curso y la terminal

Tu carpeta del curso se crea en tus Documentos y lleva tu matrícula y tu nombre, por ejemplo `C:/Users/Ana/Documents/20261234-ana-perez/` . Dentro queda `reqflow-workspace/` , donde clonarás los repositorios de los flujos que elijas. Usa esa carpeta con matrícula y nombre durante todo el taller. No necesitas clonar `bcp-ai-labs` : es el código del portal. No ejecutes los productos ni clones repositorios dentro de él. Conserva cualquier trabajo existente.

Ejecuta el instalador con la opción `curso` . Este comando crea tu carpeta del curso y, dentro, la carpeta `reqflow-workspace/` . La terminal te pide tu matrícula (de 3 a 20 letras, números o guiones) y tu nombre con acentos si los tiene. Si prefieres no responder preguntas o usar otra carpeta base en vez de Documentos, cambia a la pestaña **Avanzado** del bloque.

```
>_ Terminal · bash                                    Básico  Avanzado

curl -fsSL https://bcp-ai-labs.pages.dev/install.sh | bash -s -- curso

Numero de matricula: 20261234
Nombre completo: Ana Pérez
```

Resultado esperado: en Documentos tienes esta estructura, con tu matrícula y tu nombre. La carpeta `reqflow-workspace/` queda vacía hasta que clones los repositorios de tus flujos:

```
text

C:/Users/Ana/Documents/20261234-ana-perez/
└── reqflow-workspace/
```

> Esto es lo que hace `curso`, y puedes repetirlo sin perder nada:
>
> - Crea la carpeta con `reqflow-workspace/` dentro y guarda tus datos en `.workshop-participant`. Si la carpeta ya pertenece a otra matrícula, se detiene sin tocarla.
> - Recuerda la ruta para que los demás comandos del taller, incluidas las terminales MCP del Día 2, usen esta misma carpeta.
> - Instala Windows Terminal si falta, agrega el perfil **Workshop - Bash** como predeterminado, conserva tus otras preferencias y respalda los archivos que modifica. Configura `CHERE_INVOKING=1` para que Bash conserve la carpeta inicial.
> - Crea el acceso directo **Workshop - Bash** en el Escritorio y en el menú Inicio. Abre Windows Terminal con Git Bash en tu carpeta del curso. Para tenerlo siempre a mano, clic derecho en él y **Anclar a la barra de tareas**; Windows no permite hacerlo por comando. `--no-shortcuts` lo omite.
> - Abre dos pestañas con ese perfil: una en tu carpeta del curso y otra en `reqflow-workspace/`. `--no-terminals` las omite.

Comprueba el resultado en una pestaña de **Workshop - Bash**. Si Windows Terminal ya estaba abierto, cierra y vuelve a abrirlo para que reconozca el perfil nuevo:

```
>_ Terminal · bash
printf 'Bash: %s\n' "$BASH_VERSION"
pwd -W
```

Debes ver una versión de Bash y la ruta de tu carpeta con tu matrícula. En Configuración de Windows Terminal, el perfil predeterminado debe ser **Workshop - Bash**. Si algo no coincide, repite `curso` con los mismos datos y abre una pestaña nueva.

2. Iniciar sesión en GitHub

En Workshop - Bash, inicia sesión con tu cuenta del taller, no con la del instructor. Elige GitHub.com, HTTPS, acepta autenticar Git con tus credenciales de GitHub e inicia por navegador:

```
gh auth login
```

Para confirmar la cuenta, ejecuta `gh auth status`. Si hay varias sesiones, `gh auth switch` permite elegir. La identidad de los commits es independiente del login; si `git config user.name` o `git config user.email` están vacíos, configúralos con tu nombre y tu correo verificado o `noreply` de GitHub. Nunca pegues tokens en prompts.

3. Validar las herramientas

Necesitas Git for Windows 2.40+, GitHub CLI 2.x con sesión iniciada, VS Code, JDK 21, Maven 3.9+ sobre Java 21, Python 3.13 y Node.js 22.12+ con npm. Un navegador permite comprobar los servicios y leer el portal. Valida todo con un solo comando, en Workshop - Bash:

```
curl -fsSL https://bcp-ai-labs.pages.dev/install.sh | bash -s -- check
--day 1
```

Solo revisa el Día 1: no instala herramientas, no modifica tu carpeta del curso y no exige Podman ni herramientas de otros días. Cada fila debe decir `OK`. Si alguna falta o falla, pide al instructor corregirla, abre una terminal nueva y repite el comando.

Si falta una herramienta, el instructor abre **Git Bash como administrador** y ejecuta:

> ⚠ **Importante: tienes que ejecutar este comando como administrador.** Abre Git Bash con **Ejecutar como administrador** antes de copiarlo.

```
curl -fsSL https://bcp-ai-labs.pages.dev/install.sh | SKIP_PODMAN=1
bash -s -- tools
```

`tools` instala las herramientas del equipo. El instalador y `check` no instalan ni comprueban extensiones de VS Code. `tools` no crea tu carpeta ni configura Workshop - Bash. Para eso usa `curso` con tu usuario habitual, como indica el paso 1. Si falla una herramienta, como Maven, la instalación queda incompleta. Después abre una terminal nueva y repite `check --day 1`. No ejecutes `all`, `day01`, `day02`, `labs`, `python-envs`, `mcp-images`, `inspector` ni `day02-mcp` para prepararlo.

4. Comprobar Copilot Agent

En VS Code inicia sesión en GitHub con la cuenta autorizada para Copilot. En **Extensiones**, busca **GitHub Copilot Chat** e instálala si falta. Hazlo con tu usuario habitual. No necesitas instalar por separado la extensión `GitHub.copilot`. Abre Copilot Chat y selecciona **Agent** en el selector de modo. Si no aparece, revisa con el instructor la licencia, las políticas de la organización y la versión de VS Code.

Cuando tengas el producto abierto, pega este pedido de solo lectura:

```text
Enumera las tres carpetas de aplicaciones visibles en este workspace.
No edites archivos ni ejecutes comandos. Indica qué archivos puedes leer
y qué acceso te falta. No supongas que una extensión instalada prueba acceso a
Agent.
```

Contrasta la respuesta con el explorador. Los permisos para leer o ejecutar herramientas se revisan en VS Code. Instalar Copilot no demuestra que Agent funcione.

Cómo empezar

El comando `curso` del paso 1 crea `reqflow-workspace/` dentro de tu carpeta del curso. Si todavía no existe, completa **Antes de empezar → 1. Crear tu carpeta del curso y la terminal**.

Clona dentro de `reqflow-workspace/` los repositorios de los flujos que elijas, como se indica en **Elige tu flujo**. Si clonas los tres, la estructura queda así:

```text
reqflow-workspace/                # carpeta de trabajo, sin .git propio
├── workshop-auth-backend/        # repositorio Git de Quarkus
├── workshop-requirements-api/    # repositorio Git de FastAPI
└── workshop-login-frontend/      # repositorio Git de Angular
```

Cada repositorio incluye `README.md`, `REGLAS.md`, `RETO.md` y `docs/GOOD_PRACTICES.md`, además de código y configuración de pruebas. Conserva las guías del laboratorio fuera de estos repositorios.

Elige tu flujo

Elige la pestaña de tu proyecto para ver su carpeta y cómo iniciarlo. Puedes trabajar uno o varios flujos.

Abre **Workshop - Bash** en `reqflow-workspace/`. Si el repositorio ya está en tu equipo, entra directamente a su carpeta con el comando de la pestaña.

Si todavía no lo tienes, usa el enlace **Crear mi repositorio** de tu flujo. En GitHub, elige tu cuenta y pulsa **Create repository**. Copia la URL desde **Code**. En VS Code, pulsa **Ctrl+Shift+P**, elige **Git: Clone**, pega la URL y selecciona `reqflow-workspace/` como destino. Haz esto una sola vez.

Autenticación · Quarkus | API de requerimientos · FastAPI | Frontend · Angular

Autenticación · Java y Quarkus

Revisa el login, las credenciales y los tokens. Ver prompts.

Crear mi repositorio de autenticación

En Workshop - Bash, desde `reqflow-workspace/`, entra a `workshop-auth-backend/` y crea tu rama de trabajo:

```
Terminal · bash
```

```
cd workshop-auth-backend
git switch main
git switch -c day1/prompting
code ..
```

Si la rama ya existe, usa `git switch day1/prompting`. Dentro de este repositorio, arranca el servicio:

```
mvn quarkus:dev
```

Comprueba http://localhost:8080/q/health. Guarda los entregables en este repositorio y conserva su bloque AMBITO. Mantén los servicios en localhost y resuelve los errores antes de marcar el arranque.

Los 3 servicios

SERVICIO	PARA QUÉ SIRVE	DÓNDE COMPROBARLO
Autenticación · Quarkus	Gestiona el login y los tokens.	Salud del servicio · 8080
API de requerimientos · FastAPI	Expone los datos y las operaciones.	Documentación de la API · 8000
Frontend · Angular	Muestra la aplicación y usa los otros dos servicios.	Abrir la aplicación · 4200

Para probar el login completo, inicia los tres servicios. Si trabajas solo uno, comprueba ese servicio y anota qué integraciones quedan sin probar. Mantén los servicios en localhost.

Conserva el bloque AMBITO de tu proyecto. El agente solo puede leer y editar dentro de ese componente. Si un cambio afecta a otro, coordínalo con su responsable y comparte el contrato o la evidencia. Para cambiar ambos, el líder debe aprobar antes las rutas, los responsables y las pruebas.

Leer, copiar o descargar el material

Completado

S1 · Paso 2 de 9

Acordar reglas y roles

Lee las reglas del repositorio que trabajas y acuerda quién implementa y quién revisa.

Plantilla — REGLAS.md

Plantilla

Texto

```
# Contrato para agentes

1. Analiza [el flujo / módulo principal] completo antes de cambiar [qué].
2. Conserva el contrato de [tu endpoint o interfaz pública].
3. Crea pruebas de caracterización antes de refactorizar [qué parte crítica].
4. No silencies [tus quality gates: linter, SAST, cobertura, etc.].
5. No agregues secretos reales; usa variables de entorno y valores de prueba aislados.
6. Separa [tus capas: API / servicio / repositorio / lo que corresponda] en cambios revisables.
7. Meta de cobertura del código modificado: [tu %] o superior.

## Clean code

8. Cada función/método hace una sola cosa; si hace más de una, sepárala en capas.
9. Nombres que expliquen el propósito, no el tipo (evitar `data`, `temp`, `result2`).
10. Sin duplicación: un bloque repetido en dos lugares se extrae a algo reutilizable.
11. Evitar anidamiento profundo (`if` dentro de `if` dentro de `if`); usar retorno anticipado.
12. Sin números o strings "mágicos" sueltos; usar constantes con nombre.
```

Ejemplo real (workshop-auth-backend/REGLAS.md)

Texto

```
# Contrato para agentes

1. Analiza el flujo completo antes de cambiar autenticación.
2. Conserva el contrato de POST /auth/login.
3. Crea pruebas de caracterización antes de refactorizar persistencia o token.
4. No silencies CodeQL, Dependency Review ni JaCoCo.
5. No agregues secretos reales; usa variables de entorno y valores de prueba
aislados.
6. Separa API, servicio, repositorio y seguridad en cambios revisables.
7. Meta de cobertura del código modificado: 85% o superior.

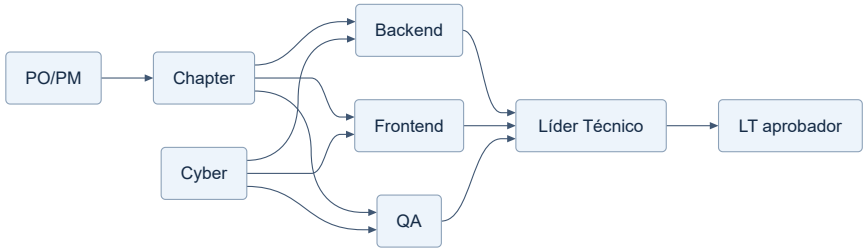
## Clean code

8. Cada método hace una sola cosa; si valida, accede a datos y arma la
   respuesta a la vez, sepáralo en capas.
9. Nombres que expliquen el propósito, no el tipo.
10. Sin duplicación: un bloque repetido se extrae a un método reutilizable.
11. Evitar anidamiento profundo; usar guard clauses / retorno anticipado.
12. Sin números o strings "mágicos" sueltos; usar constantes con nombre.
```

Cómo repartir los 8 roles según el tamaño del grupo

Acuerda quién participa en cada función y quién aprueba. Aquí se indica cuántas personas por rol según cuántos estudiantes llegan, y quién depende de quién.

Quién depende de quién



Fuente editable: roles.mmd.

PO/PM enmarca el problema de negocio → Chapter prioriza entre los 3 componentes → Backend/Frontend/QA resuelven cada uno el suyo, con Cyber revisando seguridad en los 3

→ Líder Técnico revisa que nada se rompió entre componentes → LT aprobador decide al final si el día queda aprobado (rúbrica).

6 estudiantes

ROL	PERSONAS
Backend	1
Frontend	1
QA	1
Cyber	1
Chapter + PO/PM (misma persona)	1
Líder Técnico	1
LT aprobador	el trainer

12 estudiantes

ROL	PERSONAS
Backend	2
Frontend	2
QA	2
Cyber	2
Chapter	1
PO/PM	1
Líder Técnico	1
LT aprobador	1 (o el trainer)

18 estudiantes

ROL	PERSONAS
Backend	3
Frontend	3

ROL	PERSONAS
QA	3
Cyber	2
Chapter	2
PO/PM	2
Líder Técnico	2
LT aprobador	1 (o el trainer)

Regla simple

Asigna los roles **antes** de arrancar el día, no sobre la marcha. Con grupos chicos (6), combina roles en vez de dejar alguno sin cubrir.

Leer, copiar o descargar el material

Completado

S1 · Paso 3 de 9

Construir y comprobar un prompt

Compara pedidos, adapta criterios y verifica la respuesta.

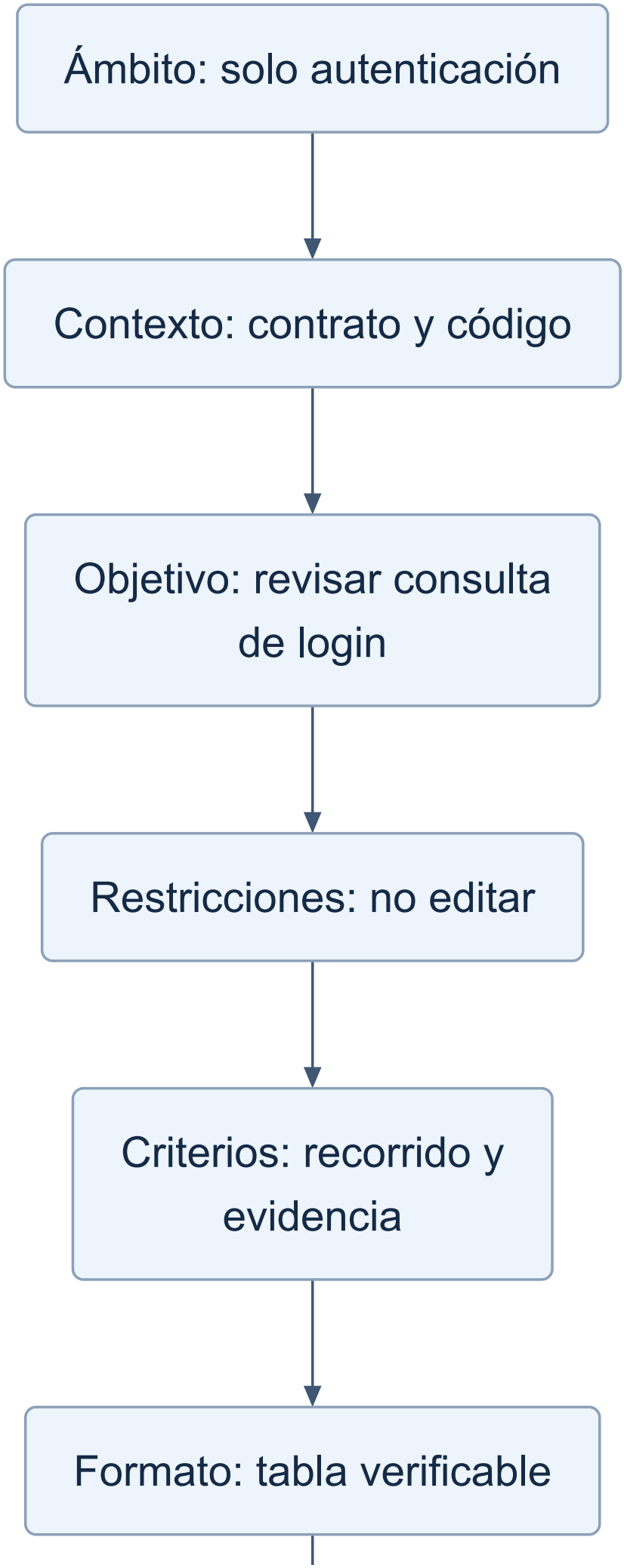
S1: construir un pedido y comprobar la respuesta

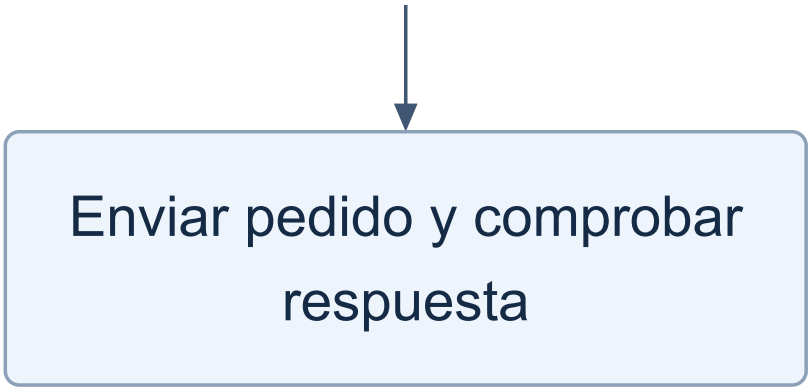
Un prompt reduce ambigüedades y hace explícitas tus decisiones. No garantiza una respuesta correcta. El agente puede leer mal el código, omitir un caso o afirmar que ejecutó una prueba que nunca corrió. Tu trabajo incluye comprobarlo.

Construir un pedido

Usaremos seis partes. Ámbito indica dónde puede actuar; las otras cinco explican qué necesita saber y qué resultado esperas.

PARTE	QUÉ APORTA	EJEMPLO DE REQFLOW
Ámbito	Delimita lectura, escritura y rutas relativas	Solo `workshop-auth-backend/`, sin tocar API ni frontend
Contexto	Aporta evidencia y decisiones existentes	Lee el contrato de `POST /auth/login`, `RETO.md` y reglas
Objetivo	Define el resultado buscado	Encontrar cómo se construye la consulta de login
Restricciones	Expresa límites al trabajo	No editar código ni cambiar el contrato
Criterios de aceptación	Permite contrastar el resultado	Citar archivo y línea, distinguir hipótesis y hallazgo confirmado
Formato de respuesta	Facilita consumir y revisar el resultado	Tabla con riesgo, evidencia y comprobación pendiente


Ámbito: solo autenticación
Contexto: contrato y código
Objetivo: revisar consulta de login
Restricciones: no editar
Criterios: recorrido y evidencia
Formato: tabla verificable



El diagrama reúne las decisiones antes de enviar el pedido. Contexto no significa pegar todo el repositorio: selecciona archivos que permitan resolver la tarea dentro del ámbito.

Comprobación: en el prompt de abajo señala qué frase limita archivos, cuál aporta contexto y cuál permite refutar la respuesta. Si una frase no tiene una función clara, reescríbela.

Comparar dos pedidos sobre la misma tarea

Pedido ambiguo, para analizar en clase antes de ejecutarlo:

```text
Revisa si el login tiene SQL injection.
```

No dice qué login, qué puede modificar ni qué evidencia necesitas. Puede producir una respuesta genérica o una corrección que no pediste. Mejora el mismo pedido:

```text
AMBITO
Trabaja unicamente en la carpeta workshop-auth-backend/ de reqflow-workspace.
Todas las rutas son relativas a esa carpeta. No leas ni modifiques otros
componentes.

CONTEXTO
ReqFlow autentica mediante POST /auth/login. Lee README.md, REGLAS.md,
RETO.md y docs/ENDPOINTS.md si existe. Localiza la ruta desde el recurso
HTTP hasta la consulta a SQLite. No des por cierta la lista de RETO.md.

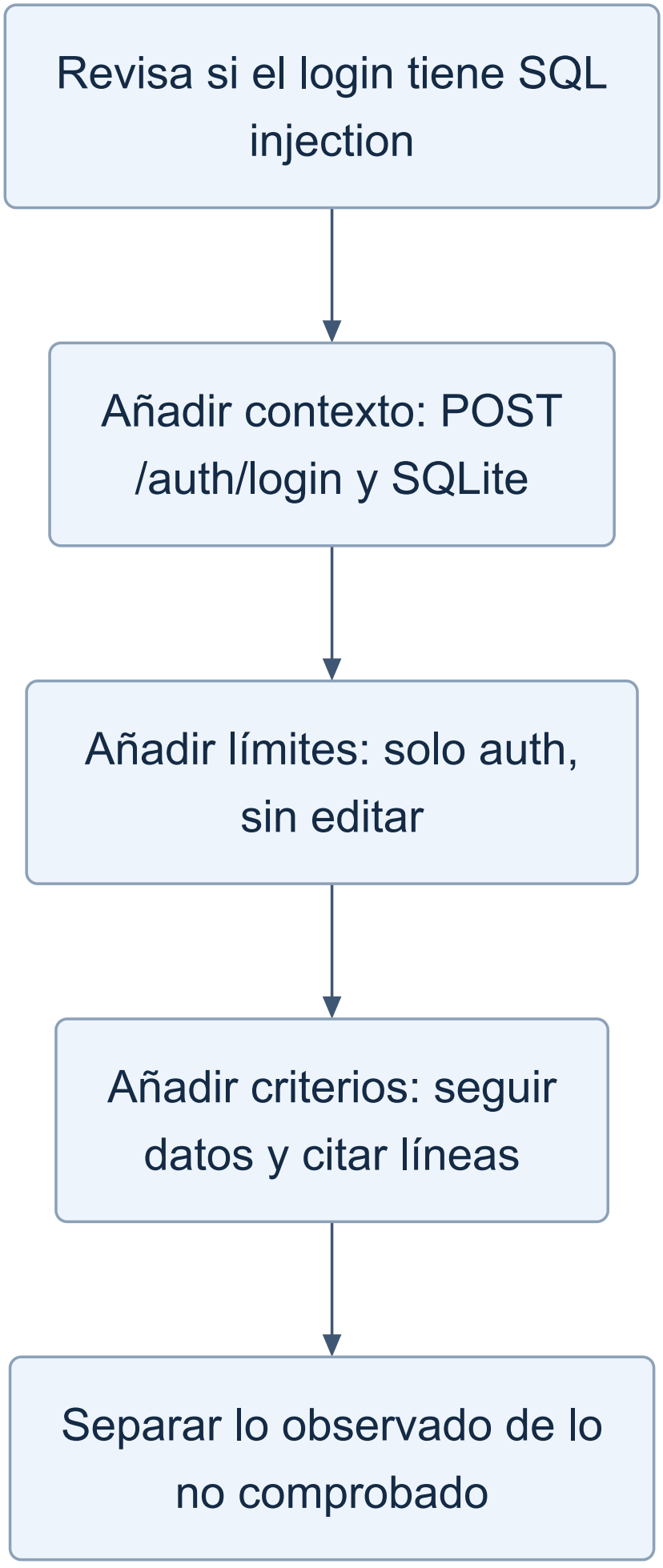
OBJETIVO
Determinar si entradas del login llegan concatenadas a una consulta SQL.
```

```
RESTRICCIONES
No edites codigo ni ejecutes ataques contra servicios remotos. Usa solo
codigo y datos ficticios locales. Si falta un archivo o no puedes ejecutar
una comprobacion, dilo. No cambies contratos ni leas otros componentes.

CRITERIOS DE ACEPTACION
Identifica el recorrido de la entrada hasta la consulta con archivo y linea.
Distingue concatenacion, parametrizacion y cualquier caso no comprobado.
Propone una prueba local con credenciales validas, invalidas y entrada con
comilla, con resultado esperado segun el contrato observado. No afirmes
que la prueba paso si solo la propusiste.

FORMATO DE RESPUESTA
```

El objetivo sigue siendo revisar SQL injection en login. Cambió la precisión: el ámbito evita mezclar implementaciones, el contexto dirige la lectura, los límites evitan ediciones prematuras y los criterios exigen una cadena de evidencia. El formato deja visibles las incertidumbres. Pedir evidencia mejora la revisión; no demuestra que las citas sean verdaderas.

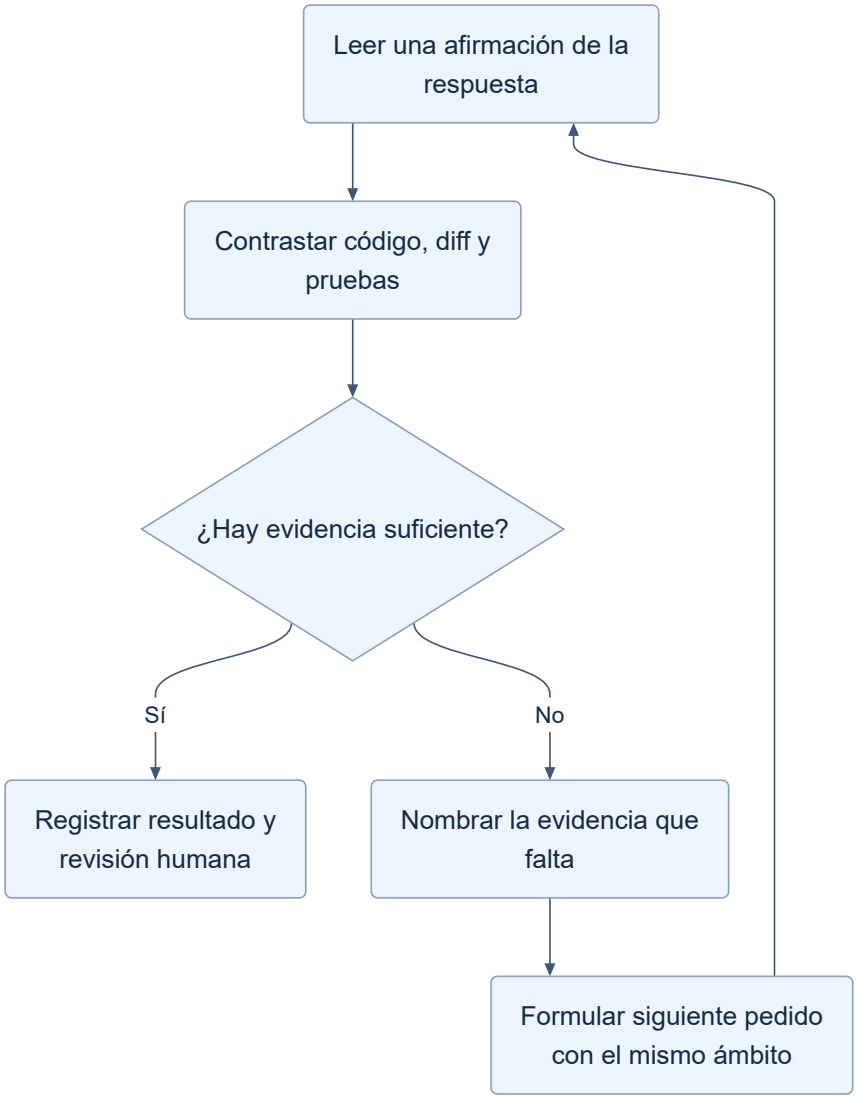

Revisa si el login tiene SQL injection
Añadir contexto: POST /auth/login y SQLite
Añadir límites: solo auth, sin editar
Añadir criterios: seguir datos y citar líneas
Separar lo observado de lo no comprobado

Cada paso elimina una ambigüedad distinta. Una respuesta más larga no prueba que el pedido mejoró: debe permitir decidir si la afirmación es correcta.

Actividad: agrega al pedido ambiguo una sola parte cada vez. Antes de enviar, escribe qué suposición deja de ser necesaria. Compara con un compañero qué información sigue faltando. No envíes el pedido ambiguo si el modo Agent tiene permisos de edición amplios.

Comprobar contra código y pruebas

1. Abre las rutas citadas dentro de `workshop-auth-backend/`. Comprueba las líneas y sigue el dato desde el request hasta la consulta. Una línea con SQL no demuestra por sí sola que sea alcanzable con entrada de usuario.
2. Lee el contrato observado y separa comportamiento existente de comportamiento deseado. Una prueba de caracterización registra el primero; una prueba de seguridad debe demostrar la corrección, aunque falle antes del cambio.
3. Ejecuta los comandos de pruebas y cobertura en tu terminal. Compara salida, código de salida y reportes con lo afirmado por el agente.
4. Revisa `git diff main` desde `reqflow-workspace/workshop-auth-backend/`. Verifica archivos nuevos con `git status --short`. Confirma que el cambio corrige la causa y que la prueba fallaría ante el defecto original.
5. Si una afirmación carece de evidencia, mantenla como hipótesis. Registra el límite y formula el siguiente pedido antes de marcar un hallazgo como resuelto.



La revisión devuelve información al prompt. Si un comando falla por el entorno, eso es un resultado de ejecución; no prueba que el código sea correcto ni que el hallazgo esté corregido.

Comprobación: toma una fila de la respuesta. ¿Otra persona puede repetir el comando y localizar las líneas sin preguntarte nada? Señala qué evidencia le falta.

Seguimiento cuando falta evidencia

```text
Conserva el AMBITO anterior: solo workshop-auth-backend/.
Afirmaste que el login usa consultas parametrizadas, pero no citaste el
punto donde se enlazan los parametros y no ejecutaste pruebas.
No edites codigo. Muestra archivo y linea de la construccion y ejecucion
de la consulta y sigue el origen de sus valores. Indica si esa evidencia
confirma o contradice tu afirmacion anterior.
Propone el test que distinguiria concatenacion de parametros enlazados.
```

```
Separa codigo leido, pruebas ejecutadas y pruebas propuestas. Si falta
acceso, nombra el bloqueo. No marques H-xx como Resuelto por esta respuesta.
```

Este seguimiento nombra la afirmación cuestionada y pide la evidencia que la resolvería. No basta con decir "sé más preciso".

Tu adaptación y su justificación

Elige un componente y una pregunta confirmable del ejercicio. Usa su plantilla, o las de API y frontend.

- Decide el foco. Por ejemplo, en API: "¿una entrada de nombre de archivo puede salir del directorio de uploads?"; en frontend: "¿se muestra un error cuando el backend no responde?".
- Completa las seis partes sin inventar rutas de código. Usa el discovery para localizar las que aún no conoces.
- Define dos criterios verificables y justifica por qué detectan el riesgo. "Código limpio" no basta. "Ante backend caído se muestra un mensaje y no se conserva sesión activa, según contrato acordado" sí puede contrastarse con una prueba.
- Pide revisión de tu prompt a un compañero. Ejecútalo después de ajustar ámbito y permisos. Contrasta una afirmación con código y una con prueba; registra lo que no pudiste comprobar.
- Guarda en `prompts/` del componente el pedido inicial, la versión adaptada, tu justificación y el seguimiento. Explica qué cambiaste después de recibir la respuesta y qué duda sigue abierta.

La plantilla contiene decisiones del estudiante, no datos que debas adivinar. Un ID como `H-02` se usa solo si existe. Un umbral debe venir de `REGLAS.md` y medirse; un formato de tabla debe ayudar a tu revisión.

Fuentes de los diagramas

Los SVG locales se generaron con Pele 1.6.0. No necesitas instalarlo para leerlos. Puedes descargar las fuentes editables construir.mmd, mejorar.mmd y revisar.mmd, o ver los SVG.

Para regenerarlos, el mantenedor instala Pele fuera del producto, por ejemplo en `output/day1/pele`, con `npm install --prefix output/day1/pele --no-package-`

`lock pele@1.6.0`, y ejecuta `node tools/day1-diagrams.mjs`. Esto no forma parte de la preparación del estudiante.

Abre el prompt del flujo que trabajas en su repositorio. En VS Code usa reqflow-workspace y conserva el bloque AMBITO para limitar al agente.

Backend · Quarkus

API · FastAPI

Frontend · Angular

Leer o descargar · Backend · Quarkus | Leer o descargar · API · FastAPI

Leer o descargar · Frontend · Angular

Completado

S2 · Paso 4 de 9

1. Encontrar hallazgos

Lee RETO.md y contrasta cada hallazgo contra el código sin modificarlo.

Abre el prompt del flujo que trabajas en su repositorio. En VS Code usa reqflow-workspace y conserva el bloque AMBITO para limitar al agente.

Backend · Quarkus

API · FastAPI

Frontend · Angular

Leer o descargar · Backend · Quarkus | Leer o descargar · API · FastAPI

Leer o descargar · Frontend · Angular

Completado

S3 · Paso 5 de 9

2. Planificar y refactorizar

Genera PLAN.md antes de corregir y actualiza HALLAZGOS.md con evidencia.

Abre el prompt del flujo que trabajas en su repositorio. En VS Code usa reqflow-workspace y conserva el bloque AMBITO para limitar al agente.

Backend · Quarkus

API · FastAPI

Frontend · Angular

Leer o descargar · Backend · Quarkus | Leer o descargar · API · FastAPI | Leer o descargar · Frontend · Angular

Completado

S2-REVIEW · Paso 6 de 9

3. Revisar las correcciones

Otra persona confirma o reabre hallazgos con un motivo comprobable.

Abre el prompt del flujo que trabajas en su repositorio. En VS Code usa reqflow-workspace y conserva el bloque AMBITO para limitar al agente.

Backend · Quarkus

API · FastAPI

Frontend · Angular

Leer o descargar · Backend · Quarkus | Leer o descargar · API · FastAPI | Leer o descargar · Frontend · Angular

Completado

S4 · Paso 7 de 9

4. Documentar el contrato final

Actualiza docs/ENDPOINTS.md con el comportamiento comprobado.

Abre el prompt del flujo que trabajas en su repositorio. En VS Code usa reqflow-workspace y conserva el bloque AMBITO para limitar al agente.

Backend · Quarkus

API · FastAPI

Frontend · Angular

Leer o descargar · Backend · Quarkus | Leer o descargar · API · FastAPI | Leer o descargar · Frontend · Angular

Completado

S4 · Paso 8 de 9

5. Completar las pruebas

Ejecuta las pruebas, cubre los riesgos y documenta resultados reales.

Abre el prompt del flujo que trabajas en su repositorio. En VS Code usa reqflow-workspace y conserva el bloque AMBITO para limitar al agente.

Backend · Quarkus

API · FastAPI

Frontend · Angular

Leer o descargar · Backend · Quarkus | Leer o descargar · API · FastAPI | Leer o descargar · Frontend · Angular

Completado Opcional

CIERRE · Paso opcional

Resolver un hallazgo adicional

Usa este prompt después del paso 2 solo si queda un H-xx por resolver. Este paso es opcional.

Abre el prompt del flujo que trabajas en su repositorio. En VS Code usa reqflow-workspace y conserva el bloque AMBITO para limitar al agente.

Backend · Quarkus

API · FastAPI

Frontend · Angular

Leer o descargar · Backend · Quarkus | Leer o descargar · API · FastAPI | Leer o descargar · Frontend · Angular

Completado

CIERRE · Paso 9 de 9

Cerrar y entregar al Día 2

Guarda código, tests, documentación y prompts; publica day1-done y comprueba el handoff.

Cierre del día: handoff al Día 2

Por componente trabajado, conserva `HALLAZGOS.md`, `PLAN.md`, `prompts/*.md`, `docs/ENDPOINTS.md` y `docs/TESTING.md`, junto con **código y tests**. Registra aprobador y revisión en `PLAN.md` o `HALLAZGOS.md`. Los IDs H-xx son locales al componente: al coordinar usa también su ruta.

Deja los pendientes reales con motivo y siguiente acción. Si resolviste todos, no inventes uno para aprobar: el instructor aporta HU de referencia al Día 2. La evaluación revisa calidad por separado del avance marcado en el portal.

Entra al repositorio del flujo que trabajaste, por ejemplo `cd workshop-auth-backend` desde `reqflow-workspace`. Repite el cierre dentro de cada repositorio trabajado. Revisa los cambios y archivos nuevos:

Terminal · bash

```
git status --short
git diff main --stat
git diff main
```

Un diff no muestra archivos sin seguimiento. Inspecciónalos también antes de agregarlos. Confirma que `.env`, `.venv`, bases SQLite, uploads, `node_modules`, `target` y reportes de cobertura no entren al commit. Ajusta `.gitignore` si hace falta. Después guarda el trabajo revisado, incluidos código, tests y documentación:

```
>_ Terminal · bash
git add -- HALLAZGOS.md PLAN.md prompts docs
# Agrega también las rutas de código y tests que modificaste.
git add -u
git diff --cached --check
git diff --cached
git commit -m 'day1: prompts, correcciones, pruebas y documentación'
git tag -a day1-done -m 'Cierre revisado del Día 1'
git push -u origin day1/prompting
git push origin refs/tags/day1-done
git ls-remote --tags origin 'refs/tags/day1-done*'
```

Revisa el staged antes de confirmar e incluye cualquier archivo raíz nuevo deliberado por nombre. El tag anotado apunta al commit de cierre de cada repositorio. Si ya existe, verifica `git show day1-done` y consulta al instructor antes de moverlo; no fuerces tags. El push explícito del tag evita depender de `--follow-tags` con tags ligeros.

Este ejercicio revisa cambios locales y publica una rama con su tag; **no exige PR ni merge**. Un PR es una solicitud de revisión antes de integrar una rama. Si el instructor lo pide, después del push ejecuta `gh pr create --base main --head day1/prompting --title 'Día 1: cambios revisados' --body 'Incluye alcance H-xx, pruebas ejecutadas y pendientes en la descripción.'`, completa esa descripción y solicita revisión humana. No confundas crear el PR con aprobarlo o integrarlo.

Descarga el verificador de handoff desde el portal y guárdalo como `check_handoff-day1.py` en tu carpeta del curso, al lado de `reqflow-workspace/`. Conserva cualquier archivo existente. Desde esa carpeta con matrícula y nombre, ejecuta en Workshop - Bash:

```
py -3.13 check_handoff-day1.py --day 1 ./reqflow-workspace
```

El verificador requiere al menos un repositorio completo con su tag `day1-done`. Solo comprueba presencia de artefactos y tag local; el push se comprueba con `git ls-remote` y la calidad con la rúbrica. Entrega enlaces a la rama y tag, rutas de componentes trabajados y pendientes.

Para los días siguientes, revisa sus requisitos: algunas actividades necesitan los tres repositorios aunque en el Día 1 hayas trabajado uno.

Evaluación D1 — Prompting + Secure AI

Puerta de entrada: ninguna (primer día). Umbral de salida: **promedio ponderado ≥ 8.0** + tag `day1-done`.

#	CRITERIO	PESO	APRUEBA (≥ 8) SI…	EVIDENCIA
1	Output	30%	Existen `HALLAZGOS.md`, `PLAN.md`, `prompts/`, `docs/ENDPOINTS.md`, `docs/TESTING.md` en cada componente trabajado; commit + tag `day1-done`	`git show day1-done`, `git ls-remote --tags origin`, `check_handoff.py --day 1` en verde
2	Contenido	30%	Pendientes reales con Id `H-xx` y siguiente acción, o resolución completa justificada con HU de referencia para D2; contrato y pruebas coherentes con el código; sin secretos ni datos reales	`HALLAZGOS.md`, diff revisado
3	Trazabilidad	20%	Cada hallazgo cita evidencia `archivo:línea`; `PLAN.md` referencia los `H-xx` que corrige	Referencias cruzadas en archivos
4	Criterio humano	20%	El review del paso 3 confirma o reabre con motivo; plan aprobado por una persona, diff local revisado y cambios comprendidos	Veredicto en `HALLAZGOS.md`

Escala por criterio: 0 ausente · 1–3 parcial · 4–5 básico · 6–7 sólido · 8–10 ejemplar y verificable.

La rama inicial es `day1/prompting` desde `main` en cada repositorio trabajado. La revisión usa `git diff main` desde el repositorio del componente, estado Git y archivos nuevos. No exige PR. El tag anotado `day1-done` se publica explícitamente después de guardar código, tests y documentación. Puedes completar uno, dos o los tres flujos; entrega evidencia de cada flujo trabajado.

Para evaluar prompting revisa el pedido ambiguo, su adaptación de seis partes, la justificación de criterios, una afirmación contrastada con código y pruebas y el seguimiento ante evidencia faltante, guardados en `prompts/` .

Leer, copiar o descargar el material